

RL Post-Training for Reasoning

GRPO, GSPO, and the DeepSeek-R1 Pipeline

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026

1. Motivation: why reasoning needs reinforcement learning
2. RL for reasoning skill refinement (verifiable rewards, process vs outcome reward models)
3. RL foundations for post-training: policies, PPO, and DPO (recap)
4. GRPO: group-relative policy optimization
5. GSPO: sequence-level optimization for long outputs and MoE models
6. DeepSeek-R1: training pipeline, rewards, emergent behavior, and key findings
7. Summary and references

- ▶ Prompting techniques and instruction tuning can elicit reasoning, but by themselves they plateau.
- ▶ The strongest reasoning models (o1/o3, DeepSeek-R1, QwQ) are made by reinforcement learning post-training: the model is rewarded for producing verifiably correct reasoning, not just plausible text.
- ▶ Classic RLHF machinery (reward models + PPO) struggles at this scale: critics double memory, reward models get hacked, and long reasoning traces amplify per-token noise.
- ▶ GRPO (critic-free, group-relative advantages, verifiable rewards) and GSPO (sequence-level optimization for long outputs and MoE models) address these problems; DeepSeek-R1 puts them together in a full training pipeline.

Reinforcement Learning for Reasoning Skill Refinement

- ▶ Reasoning tasks have a property alignment tasks lack: correctness can be *checked*. A final answer matches the key, unit tests pass, a proof term type-checks.
- ▶ **RL with verifiable rewards (RLVR)** puts that checker where the learned reward model used to be: the reward is a program, not a network.
- ▶ One step: sample a group of complete traces for a prompt, run the checker on each, then shift probability mass toward the traces that passed.
- ▶ Upside: no reward model to train or hold in memory, nothing to hack short of the checker itself, and a reward that stays correct however far the policy drifts from the SFT model.
- ▶ Downside: one bit per trace, and nothing until it finishes, so credit for a fifty-step derivation rests on the last line. The method also reaches only as far as checkers do, leaving open-ended writing and judgement calls to preference-based training.

Reinforcement Learning for Reasoning in Large Language Models with *One* Training Example

Yiping Wang^{1 † *} Qing Yang² Zhiyuan Zeng¹ Liliang Ren³ Liyuan Liu³

Baolin Peng³ Hao Cheng³ Xuehai He⁴ Kuan Wang⁵ Jianfeng Gao³

Weizhu Chen³ Shuohang Wang^{3 †} Simon Shaolei Du^{1 †} Yelong Shen^{3 †}

¹University of Washington ²University of Southern California ³Microsoft

⁴University of California, Santa Cruz ⁵Georgia Institute of Technology

Source: Wang et al. (2025)

RLVR with Very Few Training Examples (cont.)

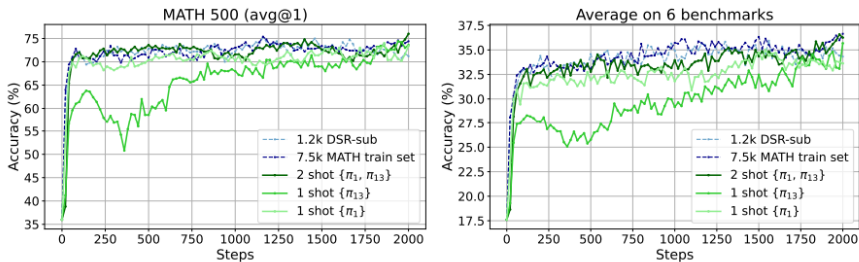


Figure 1: **RLVR with 1 example (green) can perform as well as using datasets with thousands of examples (blue).** Left/Right corresponds to MATH500/Average performance on 6 mathematical reasoning benchmarks (MATH500, AIME24, AMC23, Minerva Math, OlympiadBench, and AIME25). Base model is Qwen2.5-Math-1.5B. π_1 and π_{13} are examples defined by Eqn. 2 and detailed in Tab. 2, and they are from the 1.2k DeepScalerR subset (DSR-sub). Setup details are in Sec. 3.1. We find that RLVR with 1 example $\{\pi_{13}\}$ (35.7%) performs close to that with 1.2k DSR-sub (35.9%), and RLVR with 2 examples $\{\pi_1, \pi_{13}\}$ (36.6%) even performs better than RLVR with DSR-sub and as well as using 7.5k MATH train dataset (36.7%). Detailed results are in Fig. 6 in Appendix C.1.1. Additional results for non-mathematical reasoning tasks are in Tab. 1.

Source: Wang et al. (2025), Fig. 1.

Model's response to training example π_1 and a selected MATH500 problem. Green/Red are used for marking Correct/Wrong answers. Source: Wang et al. (2025), Fig. 3.

RLVR with Very Few Training Examples (cont.)

RL Dataset	Dataset Size	MATH 500	AIME 2024	AMC 2023	Minerva Math	Olympiad-Bench	AIME 2025	Avg.
Qwen2.5-Math-7B [24] + GRPO								
NA	NA	51.0 ^{+0.0}	12.1 ^{+0.0}	35.3 ^{+0.0}	11.0 ^{+0.0}	18.2 ^{+0.0}	6.7 ^{+0.0}	22.4 ^{+0.0}
DSR-sub	1209	78.6 ^{+27.6}	25.8 ^{+13.7}	62.5 ^{+27.2}	33.8 ^{+22.8}	41.6 ^{+23.4}	14.6 ^{+7.9}	42.8 ^{+20.4}
$\{\pi_1\}$	1	79.2 ^{+28.2}	23.8 ^{+11.7}	60.3 ^{+25.0}	27.9 ^{+16.9}	39.1 ^{+20.9}	10.8 ^{+4.1}	40.2 ^{+17.8}
$\{\pi_1, \pi_{13}\}$	2	79.2 ^{+28.2}	21.7 ^{+9.6}	58.8 ^{+23.5}	35.3 ^{+24.3}	40.9 ^{+22.7}	12.1 ^{+5.4}	41.3 ^{+18.9}
$\{\pi_1, \pi_2, \pi_{13}, \pi_{1209}\}$	4	78.6 ^{+27.6}	22.5 ^{+10.4}	61.9 ^{+26.6}	36.0 ^{+25.0}	43.7 ^{+25.5}	12.1 ^{+5.4}	42.5 ^{+20.1}
Random	16	76.0 ^{+25.0}	22.1 ^{+10.0}	63.1 ^{+27.8}	31.6 ^{+20.6}	35.6 ^{+17.4}	12.9 ^{+6.2}	40.2 ^{+17.8}
$\{\pi_1, \dots, \pi_{16}\}$	16	77.8 ^{+26.8}	30.4 ^{+18.3}	62.2 ^{+26.9}	35.3 ^{+24.3}	39.9 ^{+21.7}	9.6 ^{+2.9}	42.5 ^{+20.1}
Llama-3.2-3B-Instruct [26] + GRPO								
NA	NA	40.8 ^{+0.0}	8.3 ^{+0.0}	25.3 ^{+0.0}	15.8 ^{+0.0}	13.2 ^{+0.0}	1.7 ^{+0.0}	17.5 ^{+0.0}
DSR-sub	1209	43.2 ^{+2.4}	11.2 ^{+2.9}	27.8 ^{+2.5}	19.5 ^{+3.7}	16.4 ^{+3.2}	0.8 ^{+0.9}	19.8 ^{+2.3}
$\{\pi_1\}$	1	45.8 ^{+5.0}	7.9 ^{+0.4}	25.3 ^{+0.0}	16.5 ^{+0.7}	17.0 ^{+3.8}	1.2 ^{+0.5}	19.0 ^{+1.5}
$\{\pi_1, \pi_{13}\}$	2	49.4 ^{+8.6}	7.1 ^{+1.2}	31.6 ^{+6.3}	18.4 ^{+2.6}	19.1 ^{+5.9}	0.4 ^{+1.3}	21.0 ^{+3.5}
$\{\pi_1, \pi_2, \pi_{13}, \pi_{1209}\}$	4	46.4 ^{+5.6}	6.2 ^{+2.1}	29.1 ^{+3.8}	21.0 ^{+5.2}	15.1 ^{+1.9}	1.2 ^{+0.5}	19.8 ^{+2.3}
Qwen2.5-Math-1.5B [24] + PPO								
NA	NA	36.0 ^{+0.0}	6.7 ^{+0.0}	28.1 ^{+0.0}	8.1 ^{+0.0}	22.2 ^{+0.0}	4.6 ^{+0.0}	17.6 ^{+0.0}
DSR-sub	1209	72.8 ^{+36.8}	19.2 ^{+12.5}	48.1 ^{+20.0}	27.9 ^{+19.8}	35.0 ^{+12.8}	9.6 ^{+5.0}	35.4 ^{+17.8}
$\{\pi_1\}$	1	72.4 ^{+36.4}	11.7 ^{+5.0}	51.6 ^{+23.5}	26.8 ^{+18.7}	33.3 ^{+11.1}	7.1 ^{+2.5}	33.8 ^{+16.2}
DeepSeek-R1-Distill-Qwen-1.5B [2] + GRPO								
NA	NA	71.0 ^{+0.0}	20.0 ^{+0.0}	60.9 ^{+0.0}	24.3 ^{+0.0}	30.2 ^{+0.0}	17.9 ^{+0.0}	37.4 ^{+0.0}
DSR-sub	1209	83.4 ^{+12.4}	27.5 ^{+7.5}	80.6 ^{+19.7}	32.0 ^{+7.7}	46.5 ^{+16.3}	24.6 ^{+6.7}	49.1 ^{+11.7}
$\{\pi_1\}$	1	78.0 ^{+7.0}	25.8 ^{+5.8}	71.6 ^{+10.7}	31.2 ^{+6.9}	39.3 ^{+9.1}	20.0 ^{+2.1}	44.3 ^{+6.9}
$\{\pi_1, \pi_2, \pi_{13}, \pi_{1209}\}$	4	78.8 ^{+7.8}	29.6 ^{+9.6}	75.6 ^{+14.7}	32.0 ^{+7.7}	40.9 ^{+10.7}	23.8 ^{+5.9}	46.8 ^{+9.4}
$\{\pi_1, \dots, \pi_{16}\}$	16	82.4 ^{+11.4}	30.4 ^{+10.4}	73.4 ^{+12.5}	34.2 ^{+9.9}	42.4 ^{+12.2}	24.6 ^{+6.7}	47.9 ^{+10.5}

1(few)-shot RLVR is viable for different models and RL algorithms. Source: Wang et al. (2025), Tab. 4.

- ▶ **Outcome reward models (ORMs)** score only the final result: is the answer correct, do the tests pass?
- ▶ **Process reward models (PRMs)** score every intermediate step of the reasoning trace. Training them requires step-level labels (Lightman et al., 2023, “Let’s Verify Step by Step”, with the PRM800K dataset of human step annotations).
- ▶ PRMs give denser feedback and catch correct answers reached by invalid reasoning; ORM are cheaper, and in verifiable domains need no trained model at all, since a rule or checker suffices.
- ▶ PRMs carry their own risks: step labels are expensive, step correctness is hard to define outside mathematics, and step-level supervision can penalize unconventional but valid reasoning paths.
- ▶ DeepSeek-R1 chose rule-based outcome rewards after finding that process rewards constrained exploration and invited reward hacking.

- ▶ A **policy** π specifies how an agent (or model) chooses actions.
- ▶ **Deterministic:** $\pi(s) = a$.
- ▶ **Stochastic:** $\pi(a | s) = \Pr(a_t = a | s_t = s)$.
- ▶ For LLMs: s is the prompt/context, a is the next token; the policy is $\pi_\theta(\text{token} | \text{context})$.
- ▶ **Goal:** find $\pi^\star$ that maximizes expected cumulative reward (or, in RLHF, human-aligned reward).

- ▶ Value-based methods (Q-learning) can be hard in huge state spaces.
- ▶ Often easier to parameterize a policy class $\Pi = \{\pi_\theta \mid \theta \in \mathbb{R}^m\}$.
- ▶ Maximize expected return $\mathcal{J}(\theta) = \mathbb{E}[\sum_{t \geq 0} \gamma^t r_t \mid \pi_\theta]$.
- ▶ **Policy gradient:** update θ along $\nabla_\theta \mathcal{J}(\theta)$ (gradient ascent on parameters).
- ▶ PPO and GRPO are policy-gradient-style updates on language-model weights.

- ▶ **Problem:** large policy updates can collapse performance.
- ▶ **Idea:** take the biggest safe improvement step on the current batch of data.
- ▶ **Clipped objective** (widely used variant): with the probability ratio $\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$,

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

- ▶ ϵ is small (e.g., 0.2), which limits how far the new policy can drift per update.

- ▶ **Stable:** clipped updates keep optimization well behaved.
- ▶ Encourages exploration of phrasings at each rollout.
- ▶ **Expensive:** many rollouts, critic + RM + reference in memory.
- ▶ **Sensitive:** reward-model mistakes and distribution shift degrade it.

- ▶ Bypasses explicit RM training and RL rollouts (Rafailov et al., 2023).
- ▶ Optimizes the policy directly from preference pairs (y^+, y^-) .
- ▶ **DPO loss** (simplified form):

$$L_{\text{DPO}} = -\log \sigma \left(\beta \log \frac{\pi_{\theta}(y^+)}{\pi_0(y^+)} - \beta \log \frac{\pi_{\theta}(y^-)}{\pi_0(y^-)} \right)$$

- ▶ π_0 = frozen reference (SFT) policy; β = temperature.
- ▶ Often simpler and more stable than PPO for alignment, but still preference-based rather than driven by verifiable math or code rewards.

- ▶ **GRPO**: Group Relative Policy Optimization, a PPO-style clipped update with the critic removed.
- ▶ PPO needs a learned value network to say how good a response was *expected* to be. GRPO gets that baseline for free: sample several responses to the same prompt and score each against the group's mean reward.
- ▶ So one fewer network to train and hold in memory, and a baseline that cannot lag behind the policy because it is recomputed from scratch at every step.
- ▶ Introduced in DeepSeekMath (Shao et al., 2024), then used for the RL stages of DeepSeek-V2, DeepSeek-V3, and DeepSeek-R1.

- ▶ GRPO avoids the need for a separate critic model by estimating the baseline from group scores.
- ▶ For each question q , sample a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{\text{old}}}$.
- ▶ The policy model π_{θ} is optimized by maximizing:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(O|q)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left(w_{i,t}(\theta) \hat{A}_{i,t}, \right. \right. \right. \quad (1)$$
$$\left. \left. \left. \text{clip}(w_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t} \right) - \beta D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}}) \right\} \right]$$

- ▶ The importance ratio is formed **one token at a time**:

$$w_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})} \quad (2)$$

- KL divergence term:

$$D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})} - \log \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})} - 1 \quad (3)$$

- Under outcome supervision the checker fires once per response, so every token of o_i inherits the same advantage, computed from the group rewards $\{r_1, r_2, \dots, r_G\}$:

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})} \quad (4)$$

- ϵ and β are hyper-parameters.
- The reward is granted once, for the whole response, while the ratio and the clip act on individual tokens. That mismatch is what GSPO attacks.

- ▶ **Fewer models in memory:** policy + reference (no critic, no RM when rules suffice).
- ▶ **Often faster to train** than PPO-style RLHF on comparable hardware.
- ▶ Powers DeepSeekMath, DeepSeek-R1-Zero, and many open reproductions.
- ▶ Cheap is relative: efficient kernels move an 8B long-context run from many GPUs to one node, while 4-bit QLoRA at roughly 1 GB per billion parameters puts a 1.5B run in about 5 GB.

Llama 3.1 8B, 20K context, 8 rollouts/prompt	Standard + FA2	Optimized
Training memory	414 GB	42 GB
GRPO memory	78.3 GB	9.8 GB
Inference + KV cache	18.5 GB	2.5 GB
Total VRAM	510.8 GB	54.3 GB

Source: Unsloth, *Reinforcement Learning Guide*.

	PPO (RLHF)	GRPO (RLVR)
Baseline	Critic / value net	Mean reward in a group
Reward	Learned RM	Rules / verifiers
Models in memory	Policy, ref., RM, critic	Policy, ref. (typical)
Best for	Chat alignment	Math, code, logic

- ▶ GRPO does not replace DPO; DPO still wins when only preference pairs are available and there is no verifier.

- ▶ Long RL runs on huge models need stable updates. Unstable RL ends in model collapse that reverting to an earlier checkpoint and retuning ϵ does not recover.
- ▶ The diagnosis (Zheng et al., 2025): importance sampling corrects a distribution mismatch only when you average over many samples from the behaviour policy. GRPO applies $w_{i,t}(\theta)$ at each token position using the *single* token that was sampled there, so it corrects nothing and injects variance instead.
- ▶ That variance accumulates with response length and is amplified by per-token clipping, so the problem gets worse exactly as reasoning traces get longer.
- ▶ MoE models are worse again: on a 48-layer Qwen3-30B-A3B model, about 10% of the experts activated for the same rollout differ after one gradient update, so token-level ratios swing hard. GRPO needs a workaround, Routing Replay, which caches the old policy's routing and replays it, to converge at all.

- ▶ Rewards are usually sequence-level (correct answer, tests passed), so optimization should match that granularity.
- ▶ Same group-relative advantages as GRPO: compare G completions on the same prompt.
- ▶ Design goals ([Qwen blog](#)):
 - Stable, especially for MoE RL;
 - Efficient, with better compute-to-gain than GRPO;
 - Infrastructure-friendly, more robust to numeric precision issues in distributed RL.

- ▶ One importance ratio per response, clipped once:

$$J_{\text{GSPO}}(\theta) = \mathbb{E}_{\mathbf{x} \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | \mathbf{x})} \left[\frac{1}{G} \sum_{i=1}^G \min(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i) \right] \quad (5)$$

- ▶ The ratio is the **length-normalized sequence likelihood ratio**:

$$s_i(\theta) = \left(\frac{\pi_{\theta}(y_i | \mathbf{x})}{\pi_{\theta_{\text{old}}}(y_i | \mathbf{x})} \right)^{1/|y_i|} = \exp \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_{\theta}(y_{i,t} | \mathbf{x}, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} | \mathbf{x}, y_{i,<t})} \right) \quad (6)$$

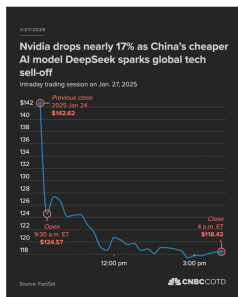
- ▶ $\hat{A}_i$ is the same group-relative advantage as GRPO. Every token of y_i now carries the same gradient weight $s_i(\theta)$ rather than its own $w_{i,t}(\theta)$, which is where the variance goes.
- ▶ The $1/|y_i|$ exponent matters: without it the ratio swings with response length, and each length would need its own ϵ .
- ▶ Because s_i and $w_{i,t}$ live on different numerical scales, ϵ does too. On Qwen3-30B-A3B the paper clips GSPO at 3×10^{-4} and 4×10^{-4} (left, right) against 0.2 and 0.27 for GRPO.

1. **Sample** a batch of prompts $x \sim \mathcal{D}$.
2. For each x , draw a **group** $\{y_i\}_{i=1}^G$ from the old policy $\pi_{\theta_{\text{old}}}$.
3. **Reward** each full response (verifier / rule checker) $\Rightarrow$ group-relative advantage $\hat{A}_i$ (same formula as GRPO).
4. Build the sequence-level importance weight $s_i(\theta)$ from $\pi_{\theta}(y_i|x)$ vs. $\pi_{\theta_{\text{old}}}(y_i|x)$.
5. **Clip and optimize once per sequence** (not token-by-token); optional KL to π_{ref} as in PPO/GRPO stacks.
6. **Variant:** GSPO-token reintroduces per-token advantages when they are needed (e.g. multi-turn RL), while keeping $s_i(\theta)$ as the clipping signal.

	GRPO	GSPO
Importance ratio	Per-token $w_{i,t}(\theta)$	Sequence-level $s_i(\theta)$
Clipping	Token-by-token	Once per sequence
Advantage	Group-relative	Group-relative (same)
Reward alignment	Mismatch on long CoT	Matches sequence rewards
MoE RL	Needs Routing Replay	Stable in Qwen3-scale runs
Typical use	DeepSeek-R1, dense RL	Qwen3, large MoE RL

- ▶ GSPO clips two orders of magnitude more tokens than GRPO and still trains more efficiently, which says the discarded token-level gradients were mostly noise.

- ▶ GSPO's clip is still hard: a response contributes either its full gradient or none of it.
- ▶ **SAPO** (Soft Adaptive Policy Optimization, Gao et al., 2025) replaces the clip with a smooth, temperature-controlled gate, so an off-policy token is attenuated rather than the whole sequence discarded.
- ▶ **HölderPO** (Chen et al., 2026) reads GRPO and GSPO as two points in one family: aggregate the token ratios with the Hölder mean of order p , where $p = 1$ (arithmetic) recovers GRPO and $p \rightarrow 0$ (geometric) recovers GSPO. Annealing p downward over training trades early signal amplification for late variance control.
- ▶ The shared lesson: what settles these algorithms is where the reward is granted and how much variance the estimator can absorb, not the shape of the loss.



Nvidia CEO Jensen Huang is impressed by the Chinese company that made Nvidia 'poorer' by \$580 billion in one single day

TOI Newscard Team / TIMESOFINDIA.COM / Updated: Jan 06, 2026, 16:37 IST

Select TOI as Comments Share AA



Nvidia's CEO Jensen Huang lauded Chinese AI startup DeepSeek for its role in advancing open-source AI, despite its model causing a massiv...

Nvidia CEO Jensen Huang has nothing but admiration for DeepSeek—the Chinese AI startup that wiped nearly \$600 billion off his company's market value in a single day last January. Speaking at the Consumer Electronics Show in Las Vegas on January 5, Huang credited DeepSeek with "activating" a global shift toward open-source artificial intelligence, calling the company's work "really, really exciting" and saying Nvidia was "so happy with it."

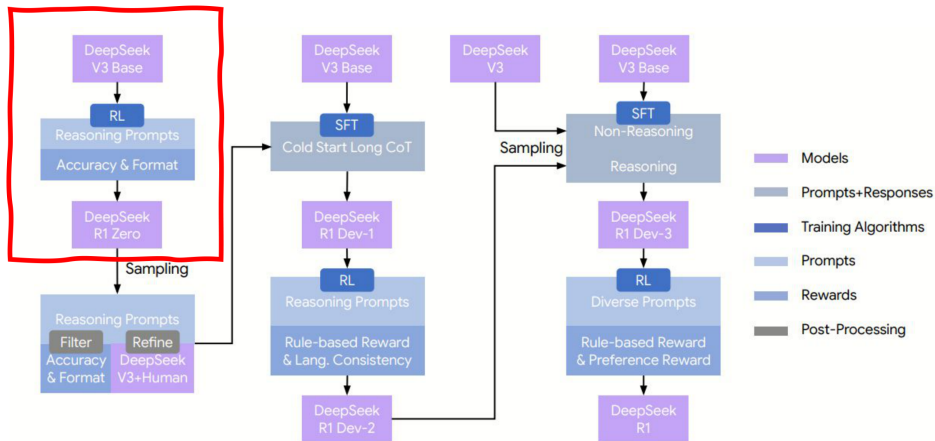
The praise comes almost a year after DeepSeek's R1 model sparked panic among investors in January 2025.

The Chinese startup claimed it had developed a competitive AI model using just 2,048 of Nvidia's older H800 chips—costing under \$6 million and taking only two months. By comparison, US companies were spending tens of millions on thousands of Nvidia's most advanced chips for similar results.

The news triggered a bloodbath on Wall Street. Nvidia's stock crashed 17% on January 27, 2025, and the

Source: Stanford CS224N, Lecture 12 (Choi, 2026); assembled from screenshots of CNBC (chart, data: FactSet) and The Times of India.

DeepSeek-R1: Training Pipeline Overview



The highlighted branch is R1-Zero: pure RL on the base model. Figure: DeepSeek-AI (2025), [arXiv:2501.12948](https://arxiv.org/abs/2501.12948), Fig. 2; red highlight added in Stanford CS224N, Lecture 12 (Choi, 2026).

Table 2 | An interesting “aha moment” of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both ...

$$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

...

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be ...

We started with the equation:

$$\sqrt{a - \sqrt{a + x}} = x$$

First, let's square both sides:

$$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$$

Next, I could square both sides again, treating the equation: ...

...

Source: DeepSeek-AI (2025), [arXiv:2501.12948](https://arxiv.org/abs/2501.12948), Table 2; via Stanford CS224N, Lecture 12 (Choi, 2026).



Accuracy Reward

For math: compare final answer against ground truth. For code: run test cases. The reward is essentially binary — correct or not.



Format Reward

A small reward for placing reasoning inside `<think>...</think>` tags and the final solution afterwards.



No Process Reward Model (PRM)

DeepSeek argues that PRMs are expensive, subject to reward hacking, and may constrain the model's ability to discover novel reasoning strategies. Outcome-based rewards give maximum freedom.

Caveat: additional composite rewards (e.g., language consistency, safety, ...) added to the later stage RL leading to the final R1

Source: Stanford CS224N, Lecture 12 (Choi, 2026).



Self-verification

The model checks its own work after deriving an answer. Emerges because self-checked outputs are more often correct.



Reflection and backtracking

“Wait, let me reconsider” or “That approach doesn’t work because...”
The model recognizes wrong paths and restarts with a different strategy.



Extended Deliberation

The response length grows from hundreds to thousands of tokens. The model is essentially learning that thinking longer leads to better answers—a learned form of test-time compute scaling.

Source: Stanford CS224N, Lecture 12 (Choi, 2026).



Poor readability

Reasoning traces are often poorly formatted and are difficult for humans to follow.



Code switching

The model frequently switches between languages within a single response, regardless of the input language.

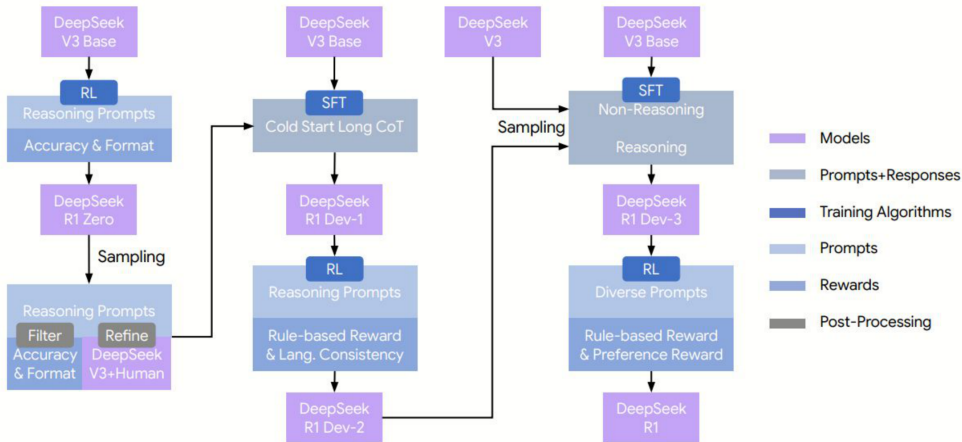


Narrow scope, no general capability or safety discovery outside RLVR

Pure RL only works well for tasks with verifiable answers (math, code). Open-ended tasks like creative writing or general conversation are not effectively addressed. Also, R1-Zero won't discover safety guardrails by solving math and code...

Source: Stanford CS224N, Lecture 12 (Choi, 2026).

DeepSeek-R1: The Full Four-Stage Pipeline



Source: DeepSeek-AI (2025), [arXiv:2501.12948](https://arxiv.org/abs/2501.12948), Fig. 2; via Stanford CS224N, Lecture 12 (Choi, 2026).

► Finding 1: RL alone can induce reasoning

- **Prior assumption:** chain-of-thought reasoning required supervised examples.
- **Finding:** R1-Zero demonstrates that outcome-based RL, with no process supervision, can induce sophisticated reasoning capabilities.
- **Caveat 1:** the final R1 is still going through the SFT→RL pipeline in the end
- **Caveat 2:** doesn't apply to small (less capable) models

Findings 1–7 follow Stanford CS224N, Lecture 12 (Choi, 2026).

► Finding 2: Outcome rewards can enable discovery

- **Prior assumption:** sophisticated process rewards are necessary for O1 reasoning.
- **Finding:** Process reward models can inadvertently constrain exploration by penalizing unconventional yet effective reasoning paths. The lesson: sometimes less supervision yields more capability.
- **Caveat:** RLVR is not applicable for all reasoning problems.

► Finding 3: RL + SFT synergy

- Neither pure RL (R1-Zero) nor pure SFT produces the best results.
- The multi-stage pipeline shows that RL and SFT play complementary roles: RL discovers capability, SFT provides reliability.
- Cold-start data prevents early RL instability; rejection sampling converts RL discoveries into reusable training data.

► Finding 4: Distillation outperforms direct RL on small models

- If you have a limited compute budget, you are better off distilling from a large reasoning model than applying RL to a small model directly.
- Through distillation, reasoning capabilities transfer to models as small as 1.5B parameters, making strong reasoning accessible at any scale.

► Finding 4: Distillation outperforms direct RL on small models

- If you have a limited compute budget, you are better off distilling from a large reasoning model than applying RL to a small model directly.
- Through distillation, reasoning capabilities transfer to models as small as 1.5B parameters, making strong reasoning accessible at any scale.

► Finding 5: Open-weight reasoning models are viable

- R1 models are released under an MIT license.
- The distilled 7B and 14B models outperform many much larger closed-source models on reasoning benchmarks.

► Finding 6: Structured search algorithms for inference scaling?

- **Prior assumption:** Inference-time search algorithm such as Monte-Carlo Tree Search might be necessary.
- **Findings:** Reasoning can be entirely autoregressive; no special search algorithm necessary.
- **Caveat:** Later models for hard math benchmarks do incorporate structure, though not in the traditional MCTS sense.

► Finding 7: Test-time compute is a learnable resource

- **Findings:** R1's models learn to allocate more thinking tokens to harder problems. Test-time compute scaling is therefore more than an inference trick like beam search or majority voting; the model itself learns when and how much to think.
- **Caveat:** true to some degree; later studies reveal challenges to address.

- ▶ **Reasoning models** spend test-time compute on chains of thought; post-training (especially RLVR) is what makes this reliable.
- ▶ **GRPO** removes the critic, uses group-relative advantages, and pairs naturally with verifiable rewards; it is the backbone of DeepSeek-R1-Zero.
- ▶ **DeepSeek-R1** combines cold-start SFT, reasoning RL, synthetic SFT, and alignment RL; distillation transfers gains to smaller models.
- ▶ **GSPO** stabilizes long-sequence and MoE RL by moving the importance ratio and the clip to the sequence level; SAPO then softens the clip into a smooth gate, and HölderPO generalizes both into one family of token aggregations.
- ▶ Plateaus, looping, and calibration issues remain open problems; scaling laws and failure modes differ from pretraining.

GSPO and successors

- ▶ Zheng, C., Liu, S., Li, M., et al. (Qwen Team, Alibaba) (2025). *Group Sequence Policy Optimization*. [arXiv:2507.18071](#) / [PDF](#).
- ▶ Qwen Team (2025). *GSPO: Towards Scalable Reinforcement Learning for Language Models* (blog). [qwen.ai/blog?id=gspos](#).
- ▶ Gao, C., Zheng, C., Chen, X.-H., et al. (Qwen Team, Alibaba) (2025). *Soft Adaptive Policy Optimization* (SAPO). [arXiv:2511.20347](#).
- ▶ Chen, Y., Liang, D., Chen, Y., et al. (2026). *Hölder Policy Optimisation*. [arXiv:2605.12058](#).

GRPO, reasoning models, and RL

- ▶ DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. [arXiv:2501.12948](#).
- ▶ Shao, Z., et al. (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models* (GRPO). [arXiv:2402.03300](#).

- ▶ Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms*. [arXiv:1707.06347](#).
- ▶ Rafailov, R., Sharma, A., Mitchell, E., et al. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. [arXiv:2305.18290](#).
- ▶ Lightman, H., Kosaraju, V., Burda, Y., et al. (2023). *Let's Verify Step by Step (PRM800K)*. [arXiv:2305.20050](#).
- ▶ Wang, Y., Yang, Q., Zeng, Z., et al. (2025). *Reinforcement Learning for Reasoning in Large Language Models with One Training Example*. [arXiv:2504.20571](#).
- ▶ Nimmaturi, D., et al. (2025). *Predictive Scaling Laws for Efficient GRPO Training of Large Reasoning Models*. [arXiv:2507.18014](#).
- ▶ OpenAI (2024). *Learning to Reason with LLMs*. [openai.com](#).
- ▶ Choi, Y. (2026). *CS224N Lecture 12: Reasoning (Part 1)*. [Stanford CS224N](#).
- ▶ CMU L3 Lab (2025). *Advanced NLP: Post-training & Meta-Generation*. [cmu-l3.github.io/anlp-spring2025](#).
- ▶ Lambert, N. *RLVR, scaling laws, reward saturation*. [natolambert.com/slides](#).

- ▶ Unsloth (2025). *Reinforcement Learning Guide* (VRAM / QLoRA + RL). unsloth.ai.
- ▶ Liu, Z., et al. (2025). *Understanding R1-Zero-Like Training: A Critical Perspective* (Dr. GRPO, length bias). [arXiv:2503.20783](https://arxiv.org/abs/2503.20783).
- ▶ Wei, J., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in LLMs*. [arXiv:2201.11903](https://arxiv.org/abs/2201.11903).
- ▶ Wang, X., et al. (2022). *Self-Consistency Improves Chain of Thought Reasoning*. [arXiv:2203.11171](https://arxiv.org/abs/2203.11171).